



## TERRAFORM CHEAT SHEET

Core setup, initialization, operations &amp; integration quick reference

PART 1: SETUP &amp; CORE

## 01. OVERVIEW &amp; INSTALL

Installation

Terraform DevOps and automation tool

```
# Install
brew install terraform
# Or via official installer
curl -L https://... | sh
terraform version
```

## 04. CORE COMMANDS

Workflow

```
terraform init      # initialize
terraform plan      # preview changes
terraform apply     # apply changes
terraform destroy   # tear down
terraform --help    # help
terraform version   # check version
```

## 02. CORE CONCEPTS

Architecture

Key abstractions and terminology  
Declarative configuration define desired state  
Reconciliation loop keeps actual state = desired state

```
# Check current state
terraform status
```

Infrastructure as Code: version control your infra

## 05. INTEGRATION &amp; CI/CD

CRUD API

Integrate with GitHub Actions / GitLab CI / Jenkins

```
# GitHub Actions example
- name: Run Terraform
  uses: terraform-actions/github@v1
  with:
    command: apply
```

Automate validation and deployment in pipelines

## 03. CONFIGURATION

YAML / Config

```
# Main config file
terraform.yaml / terraform.tf / .terraformrc
# Set environment
export TERRAFORM_HOME=/path/to/config
```

Use environment-specific configs for dev/staging/prod

```
# Validate config
terraform validate
```

## 06. SECURITY

Integration

Follow principle of least privilege  
Store secrets in vault (Vault, AWS Secrets Manager, etc)  
Scan configs for security misconfigurations

```
# Example: secret management
secret = data.vault_secret.db_password.data["..."]
Enable audit logging for all changes
```



SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. MONITORING & OBSERVABILITY

Hardening

Monitor deployment status and health

```
# Check status
terraform status
terraform logs
```

Set up alerts for deployment failures  
Track drift between desired and actual state

10. TROUBLESHOOTING

Unit Tests

```
terraform status --verbose
terraform logs --follow
```

Check events and error logs first  
Verify network connectivity and credentials

```
# Debug mode
TERRAFORM_LOG=debug terraform apply
```

08. SCALING & PERFORMANCE

Observability

Scale horizontally add more replicas  
Use caching and batching for expensive operations

```
# Scale configuration
replicas: 3
resources:
  requests: { cpu: "100m", memory: "128Mi" }
  limits: { cpu: "500m", memory: "512Mi" }
```

11. CLI REFERENCE

Commands

```
terraform --help      # full help
terraform version     # show version
terraform config       # manage config
terraform apply -auto-approve # skip prompt
terraform output       # show outputs
```

09. BEST PRACTICES

Optimization

Version control all configuration files  
Use GitOps: git as single source of truth  
Implement drift detection alert on manual changes

```
# Lint and validate before applying
terraform validate && terraform plan
```

Test in dev/staging before production

12. COMMON GOTCHAS

Warning

Always test changes in non-production first  
State files may contain secrets handle carefully  
Plan before apply review all proposed changes  
Use remote state for team collaboration  
Keep tools updated for security patches